

Task 1 Question

A short-written note or verbal explanation of the schema structure.

The scheme is for an Internship Management System.

Intern can have a mentor [Intern (M) ----- Mentor (1)]

Intern must have a track [Intern(M) ----- Track(1)]

Intern can have many projects [Intern (M) -----Project(N)]

We have created an intern_project table to represent the relationship between an intern and a project

Each submission of a project is by an intern

Task 3 Map Intern Relationships In the SQL schema:

- interns.track_id references tracks(id)
- interns.mentor_id references mentors(id)

Your task Map these relationships in Intern.

2. Questions to think about

- which relationship annotation is correct? •

which one is optional? •

which side owns the relationship?

1) @ManyToOne

Track consists of N interns

Mentor mentors N interns

2) Mentor is optional

So I have made nullable = true

3) Intern owns the relation because FK exists in Intern Table or entity

Task 8 — Analyze the intern_projects Table

The schema contains:

```
CREATE TABLE intern_projects (  
    intern_id INTEGER NOT NULL REFERENCES interns(id) ON DELETE CASCADE,  
    project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,  
    assigned_at DATE NOT NULL DEFAULT CURRENT_DATE,  
    PRIMARY KEY (intern_id, project_id)  
);
```

Your task

Decide whether this should be mapped as:

- plain @ManyToMany, or
- an explicit entity

Explain your choice.

It should be an (explicit entity).

First I have tried to make it as ManyToMany relationship but I found that it is impossible since we have assigned_at field in intern_projects, so trying to make ManyToMany relationship will remove the assigned_at field

So the solution is making explicit entity intern_projects with an embedded primary key

Task 9 — Map intern_projects

Map the assignment relationship between interns and projects.

Think about

- where assigned_at belongs
- whether a composite key is needed
- how Intern and Project connect to this table

1) I have made a new entity, assigned_at belongs to intern_projects entity

- 2) Yes a composite key is needed and it consists of (internId and projectId)
- 3) InternId references internId in intern entity, projectId references projectId in project entity

Task 11 — Map submissions

Create a `Submission` entity using the schema.

Columns involved

- `id`
- `intern_id`
- `project_id`
- `submitted_at`
- `score`

Questions

- what relationships should be mapped?
- what Java types should be used?

ManyToOne relationship for `intern_id` and `project_id`

We will have `intern` and `project` Entities inside our `Submission` entity so we can make queries easily.

I have the entities themselves joined by `@JoinColumn`

Task 16 — Optional Transaction Exercise

Create a transactional service method that:

- loads an intern
- changes the mentor
- saves the change through transaction handling

Reflection

Do you always need to call `save()` explicitly?

No, inside a transaction we don't have to write `save()` because when we open a transaction and load an entity the entity will be in the persistence layer so when we modify it, the entity in the persistence layer will be modified and after we exit the transaction it will be stored in the database

Task 17 — Constraint Awareness

From the SQL file, identify and explain the impact of:

- unique constraints
- foreign keys
- check constraints
- composite primary key

Goal

Relate schema constraints to JPA design.

- 1) Unique constrain is necessary to ensure no to rows in the database has the same value for a specific field, for example (Unique email)
- 2) Foreign Keys are very important because they allow us to implement relationships
Ex) Intern stored a FK for mentor and we can use it
To know the mentorName of an intern and mentor data
- 3) Constrains are important so we can limit or control the data stored in our database
Like being unique, Not Null, or having max value or min value or length
- 4) Primary key should be unique, we use composite primary key if using only one field doesn't give us uniqueness,
Ex) Intern_project table
We used intern_id and project id as composite primary key, and also we used each field of them to reference intern table and project table

Task 18 — Index Awareness

The SQL file creates indexes for:

- `interns(track_id)`
- `interns(mentor_id)`
- `intern_projects(project_id)`
- `projects(status)`

6

Reflection question

Why are these indexes useful for the kinds of repository methods you wrote?

Because as we learned indexes are made by default for unique fields but aren't made by default for FK

`Interns(track_id)`

`Interns(mentor_id)`

`Intern_projects(project_id)`

Help us to find `track_id`, `mentor_id` and `project_id` in less time by indexing datastructure

Note: We don't have default index for `project_id` because we have composite primary key it is not the primary key

`Project(status)` index will let us search project by status faster without looping over all projects